ECE 2011
Introduction to Computer Engineering
Spring 2026

Lecture 12
02-19-2026

Linghao Song
Yale ECE

Some Logistics

- Labs:
  - Lab 1 – 6: one-week session
  - Lab 7: two-week session
  - Lab Practical Exam: April 8/10
- Feb 24 Tuesday Schedule
  - No in-class lecture, watch two videos
    - Architecture All Access: Modern FPGA Architecture, https://www.youtube.com/watch?v=EVy4KEj9kZg
    - Architecture All Access: Modern CPU Architecture Part 1 – Key Concepts, https://www.youtube.com/watch?v=vgPFzblBh7w
    - Bonus(2%): submit a 1-page summary to Canvas
- Problem Set 1:
  - March 5
  - Boolean Minimization, Finite State Machine, Timing Analysis, Understanding Verilog
  - Prepare your cheat sheet: up to 4 pages, submit with solutions on Cavas

Digital Building Blocks

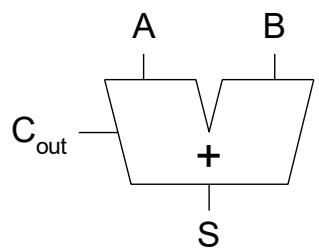
- **Digital building blocks:**
  - Gates, multiplexers, decoders, registers, arithmetic circuits, counters, memory arrays, logic arrays
- **Building blocks demonstrate hierarchy, modularity, and regularity:**
  - Hierarchy of simpler components
  - Well-defined interfaces and functions
  - Regular structure easily extends to different sizes
- **We'll use these building blocks to build a microprocessor**

© *Digital Design and Computer Architecture*, 2nd Edition, 2012

ELSEVIER

1-Bit Adders

Half Adder

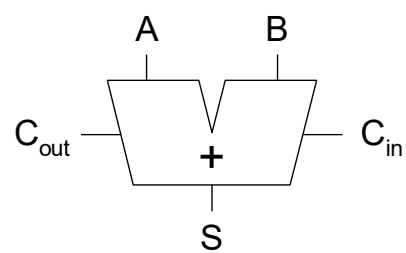


A	B	C_{out}	S
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

$S = A \oplus B$

$C_{out} = AB$

Full Adder



C_{in}	A	B	C_{out}	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

$S = A \oplus B \oplus C_{in}$

$C_{out} = AB + AC_{in} + BC_{in}$

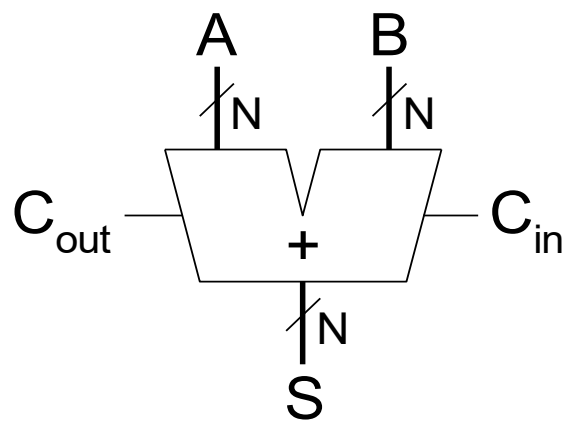
© *Digital Design and Computer Architecture*, 2nd Edition, 2012

ELSEVIER

Multibit Adders (CPAs)

- Types of carry propagate adders (CPAs):
  - **Ripple-carry** (slow)
  - **Carry-lookahead** (fast)
  - **Prefix** (faster)
- Carry-lookahead and prefix adders faster for large adders but require more hardware

Symbol

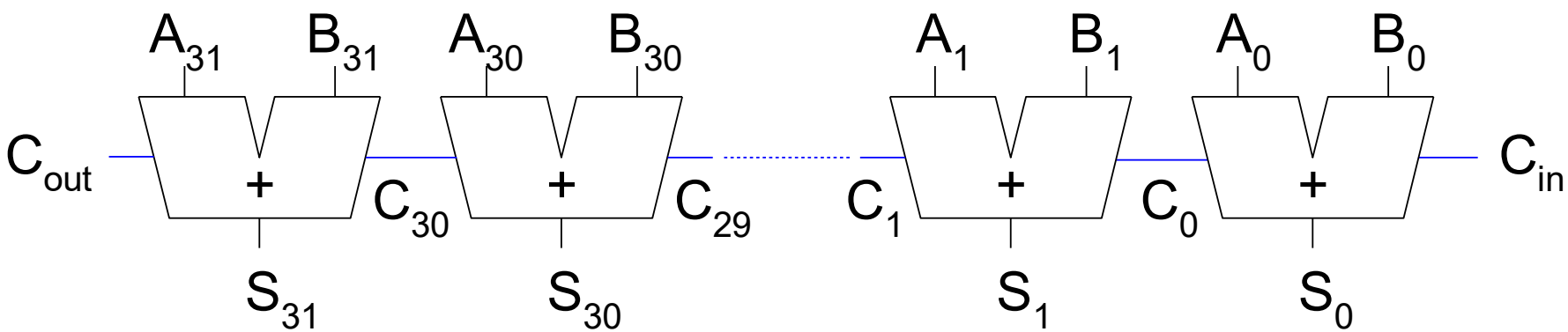


© *Digital Design and Computer Architecture*, 2nd Edition, 2012

ELSEVIER

Ripple-Carry Adder

- Chain 1-bit adders together
- Carry ripples through entire chain
- Disadvantage: **slow**



© *Digital Design and Computer Architecture*, 2nd Edition, 2012

Ripple-Carry Adder Delay

$$t_{\text{ripple}} = Nt_{FA}$$

where t_{FA} is the delay of a 1-bit full adder

© *Digital Design and Computer Architecture*, 2nd Edition, 2012

ELSEVIER

Carry-Lookahead Adder

- Compute carry out (C_{out}) for k-bit blocks using *generate* and *propagate* signals
- **Some definitions:**
  - Column i produces a carry out by either ***generating*** a carry out or ***propagating*** a carry in to the carry out
  - Generate (G_i) and propagate (P_i) signals for each column:
    - **Generate:** Column i will generate a carry out if A_i AND B_i are both 1.

$$G_i = A_i B_i$$

    - **Propagate:** Column i will propagate a carry in to the carry out if A_i OR B_i is 1.

$$P_i = A_i + B_i$$

    - **Carry out:** The carry out of column i (C_i) is:

$$C_i = A_i B_i + (A_i + B_i)C_{i-1} = G_i + P_i C_{i-1}$$

© *Digital Design and Computer Architecture*, 2nd Edition, 2012

ELSEVIER

Block Propagate & Generate Signals

Now use column Propagate and Generate signals to compute ***Block Propagate*** and ***Block Generate*** signals for k-bit blocks, i.e.:

- Compute if a k-bit group will **propagate** a carry in (of the block) to the carry out (of the block)
- Compute if a k-bit group will **generate** a carry out (of the block)

$\frac{N}{k}$ blocks in total.

© *Digital Design and Computer Architecture*, 2nd Edition, 2012

ELSEVIER

Block Propagate & Generate Signals

- **Example:** 4-bit blocks
  - Block propagate signal: $\boldsymbol{P_{3:0}}$ (single-bit signal)
  - Block generate signal: $\boldsymbol{G_{3:0}}$ (single-bit signal)

© *Digital Design and Computer Architecture*, 2nd Edition, 2012

ELSEVIER

Block Propagate Signal

- **Example:** 4-bit blocks
  - Block propagate signal: $P_{3:0}$ (single-bit signal)
    - A carry-in would propagate through all 4 bits of the block:

$$P_{3:0} = P_3 P_2 P_1 P_0$$

© *Digital Design and Computer Architecture*, 2nd Edition, 2012

ELSEVIER

Block Propagate Signal Examples

- **Example:** 4-bit blocks
  - Block propagate signal: $P_{3:0}$ (single-bit signal)
    - A carry-in would propagate through all 4 bits of the block:

$$P_{3:0} = P_3 P_2 P_1 P_0$$

    - **Examples:**

1011	$A_{3:0}$
+ 0100	$B_{3:0}$
1111	P_3, P_2, P_1, P_0
0000	G_3, G_2, G_1, G_0

$$\boxed{P_{3:0} = P_3 P_2 P_1 P_0 = 1}$$

1011	$A_{3:0}$
+ 0001	$B_{3:0}$
1011	P_3, P_2, P_1, P_0
0001	G_3, G_2, G_1, G_0

$$\boxed{P_{3:0} = P_3 P_2 P_1 P_0 = 0}$$

© *Digital Design and Computer Architecture*, 2nd Edition, 2012

ELSEVIER

Block Propagate Signal Examples

- **Example:** 4-bit blocks
  - Block propagate signal: $P_{3:0}$ (single-bit signal)
    - A carry-in would propagate through all 4 bits of the block:

$$P_{3:0} = P_3 P_2 P_1 P_0$$

    - **Examples:**

Carry-out (C_3) Carry-in (C_{-1})

11111	$C_3, C_2, C_1, C_0, C_{-1}$
1011	$A_{3:0}$
+ 0100	$B_{3:0}$
0000	$S_{3:0}$
1111	P_3, P_2, P_1, P_0
0000	G_3, G_2, G_1, G_0

$P_{3:0} = P_3 P_2 P_1 P_0 = 1$

Carry-out (C_3) Carry-in (C_{-1})

00111	$C_3, C_2, C_1, C_0, C_{-1}$
1011	$A_{3:0}$
+ 0001	$B_{3:0}$
1101	$S_{3:0}$
1011	P_3, P_2, P_1, P_0
0001	G_3, G_2, G_1, G_0

$P_{3:0} = P_3 P_2 P_1 P_0 = 0$

© Digital Design and Computer Architecture, 2nd Edition, 2012

ELSEVIER

Block Propagate & Generate Signals

- **Example:** 4-bit blocks
  - Block propagate signal: $\boldsymbol{P_{3:0}}$ (single-bit signal)
    - A carry-in would propagate through all 4 bits of the block:

$$P_{3:0} = P_3 P_2 P_1 P_0$$

  - Block generate signal: $\boldsymbol{G_{3:0}}$ (single-bit signal)

© *Digital Design and Computer Architecture*, 2nd Edition, 2012

ELSEVIER

Block Propagate & Generate Signals

- **Example:** 4-bit blocks
  - Block propagate signal: $P_{3:0}$ (single-bit signal)
    - A carry-in would propagate through all 4 bits of the block:

$$P_{3:0} = P_3 P_2 P_1 P_0$$

  - Block generate signal: $G_{3:0}$ (single-bit signal)
    - A carry is generated:
      - in column 3, **or**
      - in column 2 and propagated through column 3, **or**
      - in column 1 and propagated through columns 2 and 3, **or**
      - in column 0 and propagated through columns 1-3

© *Digital Design and Computer Architecture*, 2nd Edition, 2012

ELSEVIER

Block Propagate & Generate Signals

- **Example:** 4-bit blocks
  - Block propagate signal: $P_{3:0}$ (single-bit signal)
    - A carry-in would propagate through all 4 bits of the block:

$$P_{3:0} = P_3 P_2 P_1 P_0$$

  - Block generate signal: $G_{3:0}$ (single-bit signal)
    - A carry is generated:
      - in column 3, **or**
      - in column 2 and propagated through column 3, **or**
      - in column 1 and propagated through columns 2 and 3, **or**
      - in column 0 and propagated through columns 1-3

$$G_{3:0} = G_3 + G_2 P_3 + G_1 P_2 P_3 + G_0 P_1 P_2 P_3$$

© *Digital Design and Computer Architecture*, 2nd Edition, 2012

ELSEVIER

Block Generate Signal Examples

- **Example:** 4-bit blocks
  - Block generate signal: $G_{3:0}$ (single-bit signal)
    - A carry is: generated in column 3, **or** generated in column 2 and propagated through column 3, **or** ...

$$G_{3:0} = G_3 + G_2P_3 + G_1P_2P_3 + G_0P_1P_2P_3$$

    - **Examples:**

1001	$A_{3:0}$	1110	$A_{3:0}$	0110	$A_{3:0}$
+ 1100	$B_{3:0}$	+ 0100	$B_{3:0}$	+ 0010	$B_{3:0}$
1101	P_3,P_2,P_1,P_0	1110	P_3,P_2,P_1,P_0	0110	P_3,P_2,P_1,P_0
1000	G_3,G_2,G_1,G_0	0100	G_3,G_2,G_1,G_0	0010	G_3,G_2,G_1,G_0
$G_{3:0} = 1$		$G_{3:0} = 1$		$G_{3:0} = 0$	

© *Digital Design and Computer Architecture*, 2nd Edition, 2012

ELSEVIER

Block Generate Signal Examples

- **Example:** 4-bit blocks
  - Block generate signal: $G_{3:0}$ (single-bit signal)
    - A carry is: generated in column 3, **or** generated in column 2 and propagated through column 3, **or** ...

$$G_{3:0} = G_3 + G_2P_3 + G_1P_2P_3 + G_0P_1P_2P_3$$

    - **Examples:**

Carry-out (C_3) ↓		Carry-out (C_3) ↓		Carry-out (C_3) ↓	
10011	C_3,C_2,C_1,C_0,C_{-1}	11000	C_3,C_2,C_1,C_0,C_{-1}	01101	C_3,C_2,C_1,C_0,C_{-1}
1001	$A_{3:0}$	1110	$A_{3:0}$	0110	$A_{3:0}$
+ 1100	$B_{3:0}$	+ 0100	$B_{3:0}$	+ 0010	$B_{3:0}$
0110	$S_{3:0}$	0010	$S_{3:0}$	1000	$S_{3:0}$
1101	P_3,P_2,P_1,P_0	1110	P_3,P_2,P_1,P_0	0110	P_3,P_2,P_1,P_0
1000	G_3,G_2,G_1,G_0	0100	G_3,G_2,G_1,G_0	0010	G_3,G_2,G_1,G_0
$G_{3:0} = 1$		$G_{3:0} = 1$		$G_{3:0} = 0$	

© *Digital Design and Computer Architecture*, 2nd Edition, 2012

ELSEVIER

Carry-Lookahead Adder

- **Example:** 4-bit blocks
  - Block propagate signal: $P_{3:0}$ (single-bit signal)
    - A carry-in would propagate through all 4 bits of the block:

$$P_{3:0} = P_3 P_2 P_1 P_0$$

  - Block generate signal: $G_{3:0}$ (single-bit signal)
    - A carry is generated:
      - in column 3, **or**
      - in column 2 and propagated through column 3, **or** …

$$G_{3:0} = G_3 + G_2 P_3 + G_1 P_2 P_3 + G_0 P_1 P_2 P_3$$

$$G_{3:0} = G_3 + P_3 [G_2 + P_2 (G_1 + P_1 G_0)]$$

$$C_3 = G_{3:0} + P_{3:0} C_{-1}$$

© *Digital Design and Computer Architecture*, 2nd Edition, 2012

ELSEVIER

Carry-Lookahead Adder

- **Example:** 4-bit blocks ($G_{3:0}$ and $P_{3:0}$) :

$$G_{3:0} = G_3 + P_3\,[G_2 + P_2\,(G_1 + P_1 G_0)]$$

$$P_{3:0} = P_3 P_2 P_1 P_0$$

$$C_3 = G_{3:0} + P_{3:0}\,C_{-1}$$

- **Generally,**

$$G_{i:j} = G_i + P_i\,[G_{i-1} + P_{i-1}\,(G_{i-2} + P_{i-2} G_j)]$$

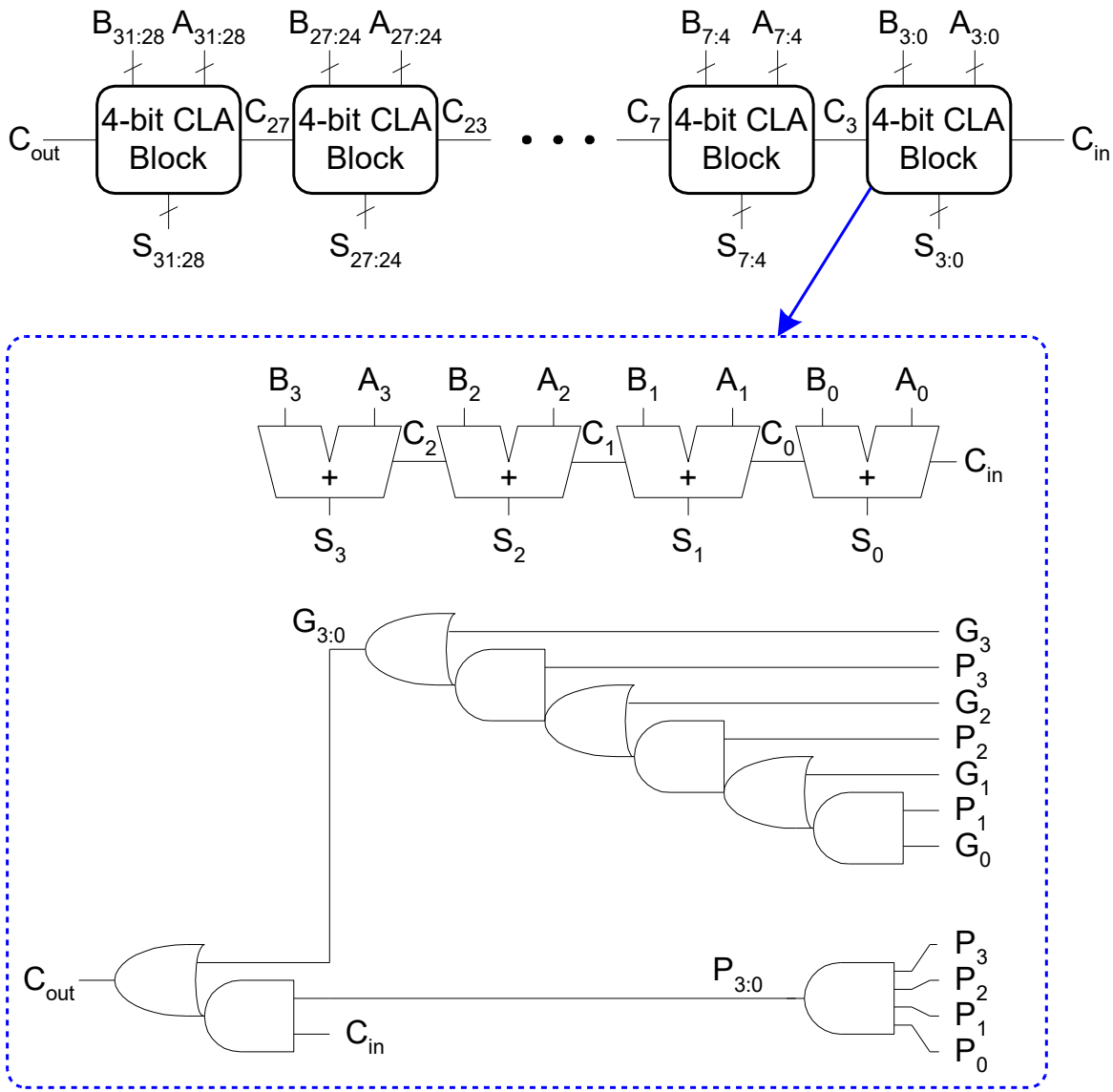
$$P_{i:j} = P_i P_{i-1} P_{i-2} P_j$$

$$C_i = G_{i:j} + P_{i:j}\,C_{j-1}$$

© *Digital Design and Computer Architecture*, 2nd Edition, 2012

ELSEVIER

32-bit CLA with 4-bit Blocks



© *Digital Design and Computer Architecture*, 2nd Edition, 2012

ELSEVIER

Carry-Lookahead Addition

- **1:** Compute G_i and P_i for all columns

$$G_i = A_i B_i$$

$$P_i = A_i + B_i$$

© *Digital Design and Computer Architecture*, 2nd Edition, 2012

ELSEVIER

Carry-Lookahead Addition

- **1:** Compute G_i and P_i for all columns
- **2:** Compute G and P for k-bit blocks

$$P_{3:0} = P_3 P_2 P_1 P_0$$

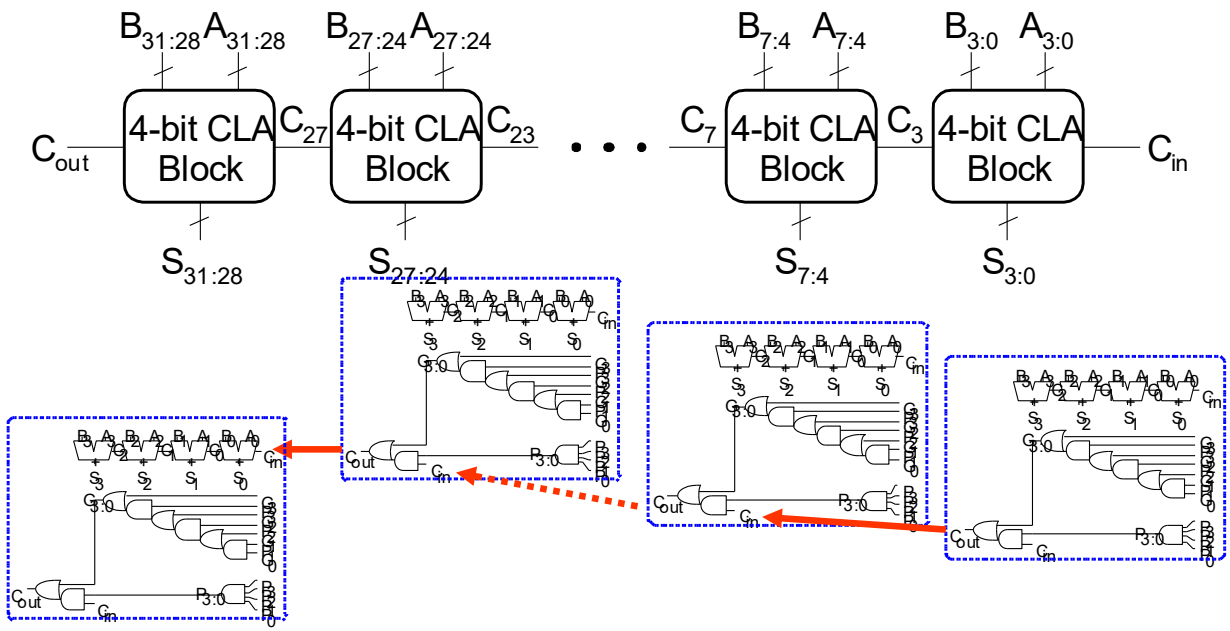
$$G_{3:0} = G_3 + P_3 (G_2 + P_2 (G_1 + P_1 G_0))$$

© *Digital Design and Computer Architecture*, 2nd Edition, 2012

ELSEVIER

Carry-Lookahead Addition

- **1:** Compute G_i and P_i for all columns
- **2:** Compute G and P for k-bit blocks
- **3:** C_{in} propagates through each k-bit propagate/generate logic (meanwhile computing sums)

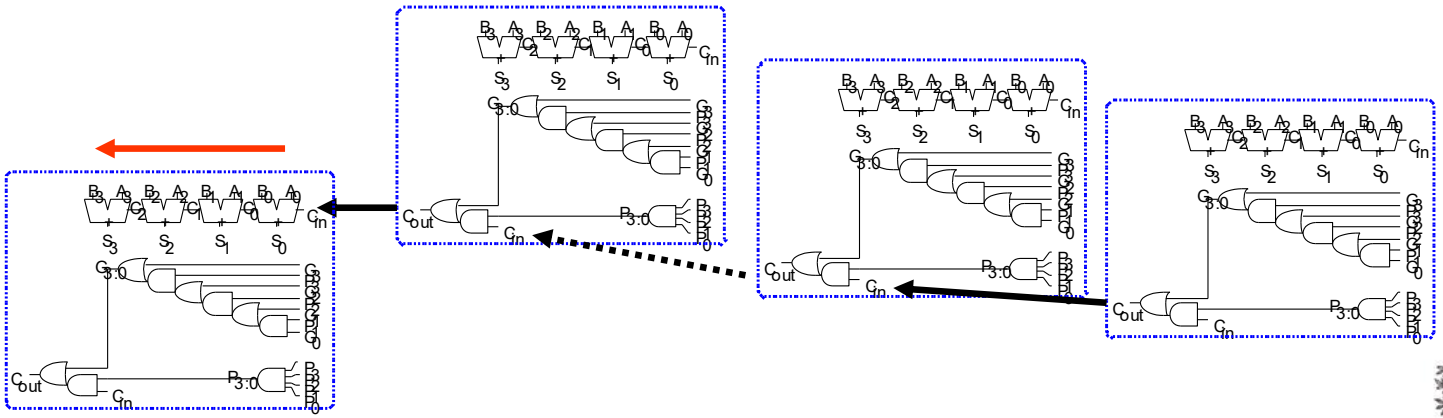


© *Digital Design and Computer Architecture*, 2nd Edition, 2012

ELSEVIER

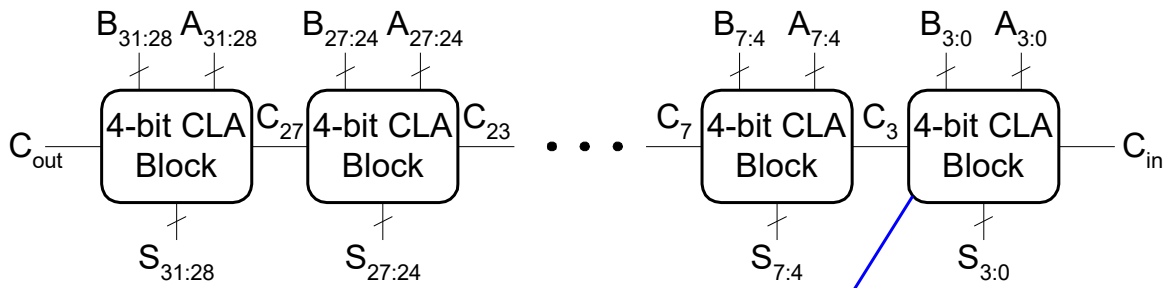
Carry-Lookahead Addition

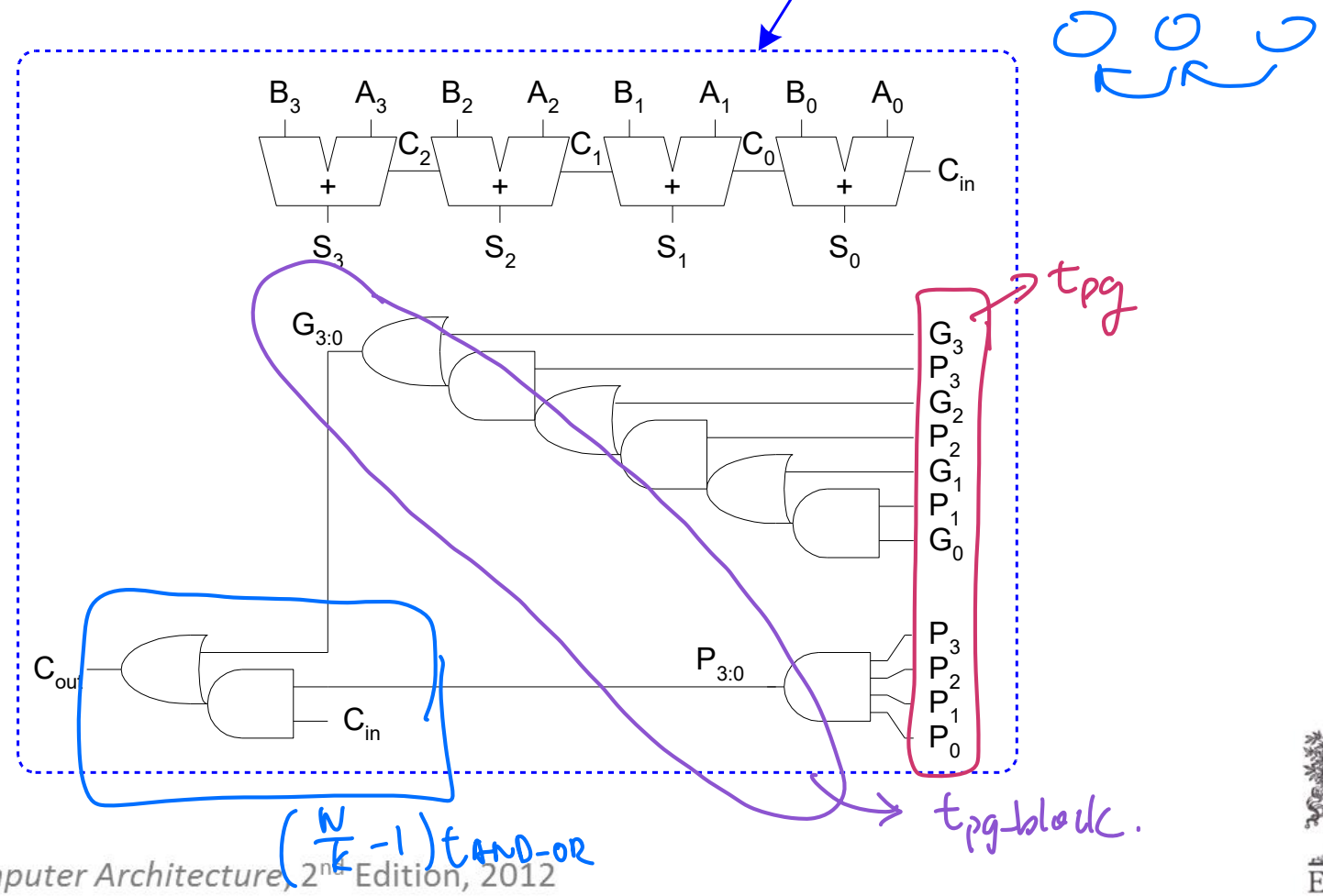
- **1:** Compute G_i and P_i for all columns
- **2:** Compute G and P for k-bit blocks
- **3:** C_{in} propagates through each k-bit propagate/generate logic (meanwhile computing sums)
- **4:** Compute sum for k-bit blocks



© *Digital Design and Computer Architecture*, 2nd Edition, 2012

32-bit CLA with 4-bit Blocks





© Digital Design and Computer Architecture, 2nd Edition, 2012

ELSEVIER

Carry-Lookahead Adder Delay

For N-bit CLA with k-bit blocks:

$$t_{CLA} = t_{pg} + t_{pg_block} + (N/k - 1)t_{AND_OR} + kt_{FA}$$

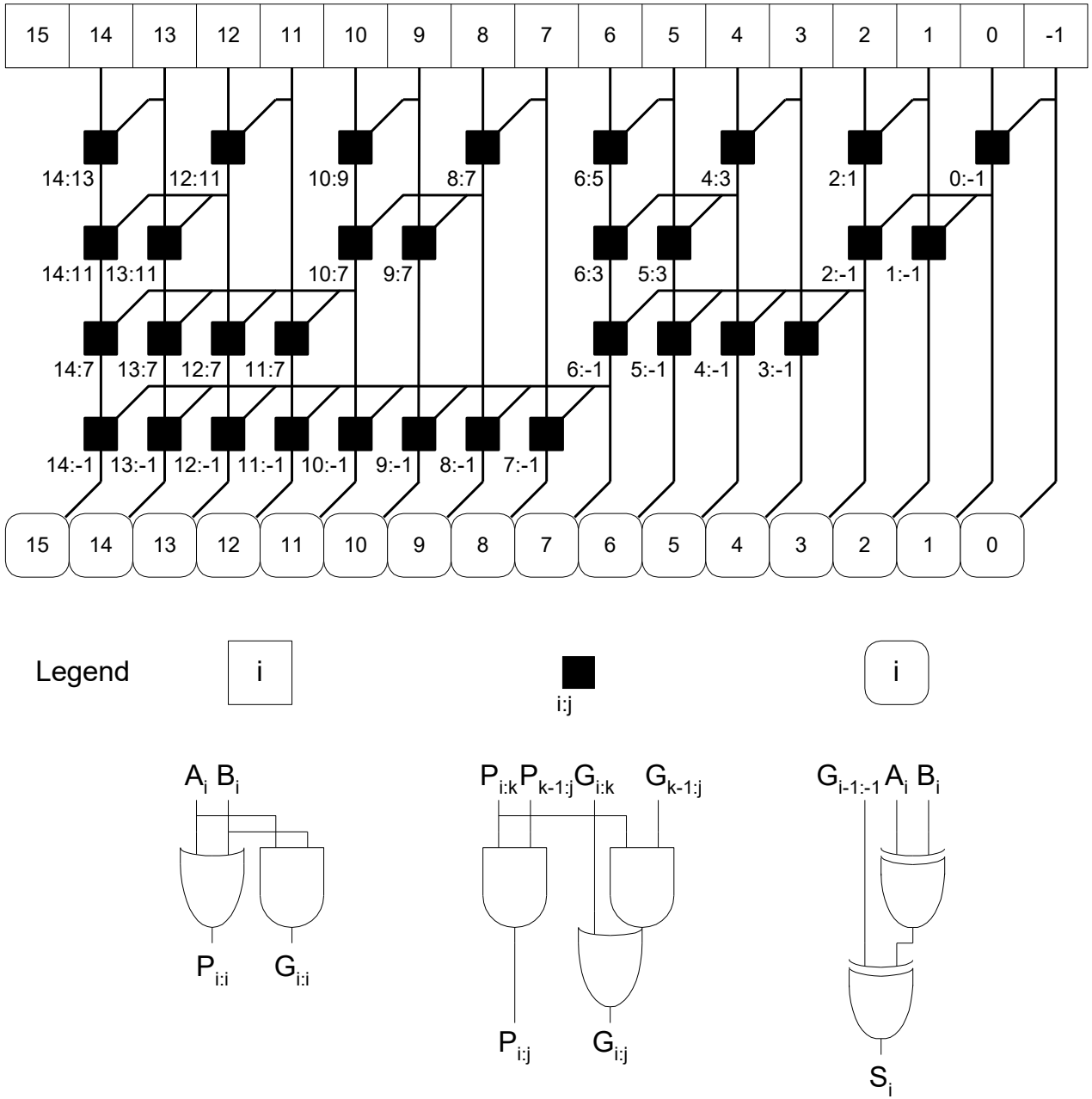
- t_{pg} : delay to generate all P_i, G_i
- t_{pg_block} : delay to generate all $P_{i:j}$, $G_{i:j}$
- t_{AND_OR} : delay from C_{in} to C_{out} of final AND/OR gate in k-bit CLA block

An N-bit carry-lookahead adder is generally much faster than a ripple-carry adder for $N > 16$

© *Digital Design and Computer Architecture*, 2nd Edition, 2012

ELSEVIER

Prefix Adder Schematic



© *Digital Design and Computer Architecture*, 2nd Edition, 2012

ELSEVIER

Prefix Adder Delay

$$t_{PA} = t_{pg} + \log_2 N(t_{pg_prefix}) + t_{XOR}$$

- t_{pg}: delay to produce P_i G_i (AND or OR gate)
- t_{pg_prefix}: delay of black prefix cell (AND-OR gate)

© *Digital Design and Computer Architecture*, 2nd Edition, 2012

ELSEVIER

Adder Delay Comparisons

Compare delay of: 32-bit ripple-carry, carry-lookahead, and prefix adders

- CLA has 4-bit blocks
- 2-input gate delay = 100 ps; full adder delay = 300 ps

$$t_{\text{ripple}} = Nt_{FA} = 32(300 \text{ ps})$$
$$= \mathbf{9.6\ ns}$$

$$t_{CLA} = t_{pg} + t_{pg_\text{block}} + (N/k - 1)t_{\text{AND_OR}} + kt_{FA}$$
$$= [100 + 600 + (7)200 + 4(300)] \text{ ps}$$
$$= \mathbf{3.3\ ns}$$

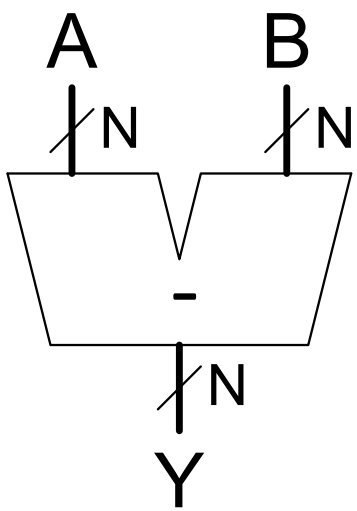
$$t_{PA} = t_{pg} + \log_2 N(t_{pg_\text{prefix}}) + t_{\text{XOR}}$$
$$= [100 + \log_2 32(200) + 100] \text{ ps}$$
$$= \mathbf{1.2\ ns}$$

© *Digital Design and Computer Architecture*, 2nd Edition, 2012

ELSEVIER

Subtracter

Symbol



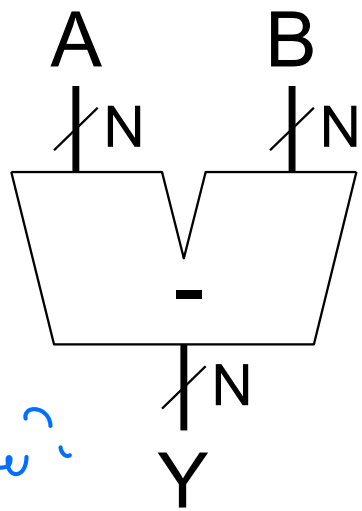
Subtracter

00000 0
1 0 1 1 1 (-7)

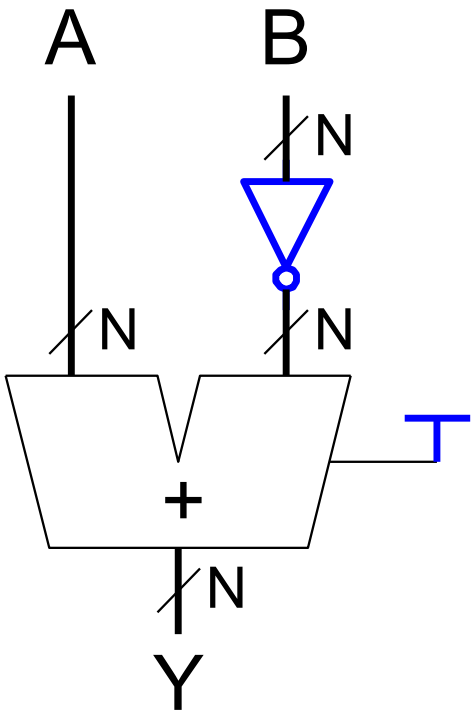
1 0 1 1 1

Does the inverter invert everything here?

Symbol



Implementation

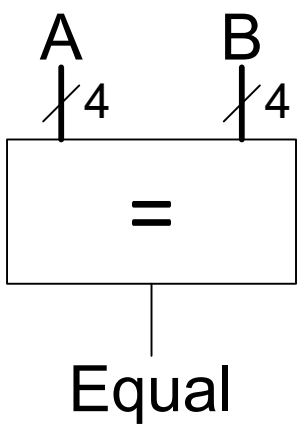


© *Digital Design and Computer Architecture*, 2nd Edition, 2012

ELSEVIER

Comparator: Equality

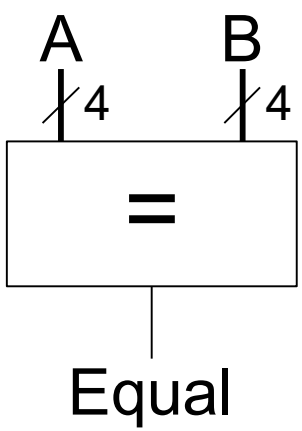
Symbol



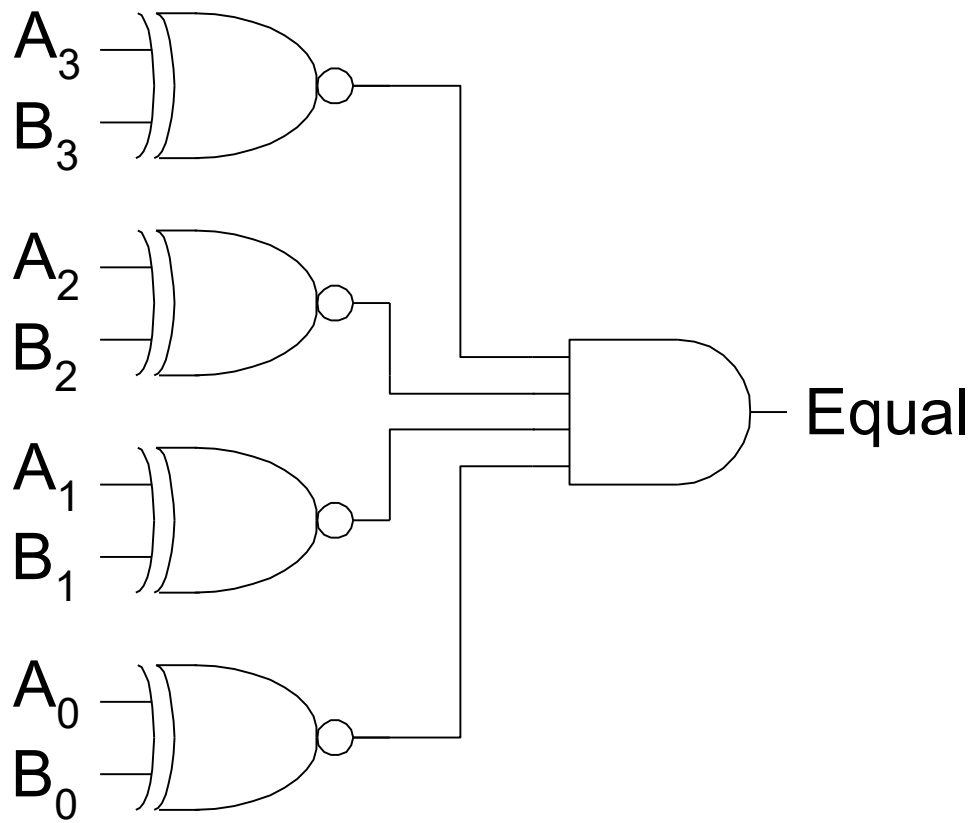
© *Digital Design and Computer Architecture*, 2nd Edition, 2012

Comparator: Equality

Symbol

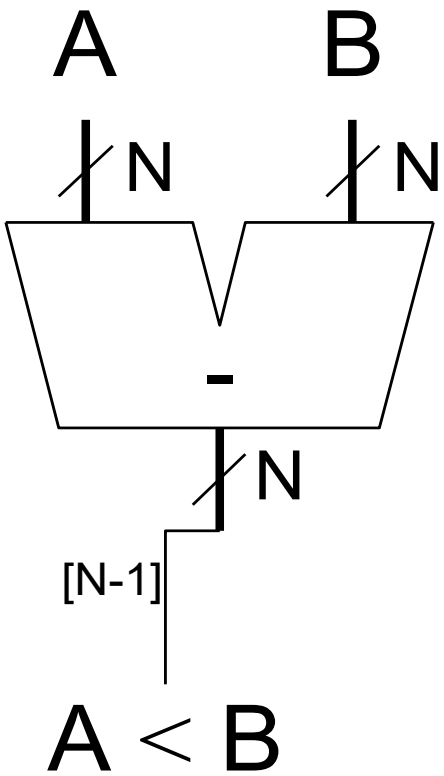


Implementation



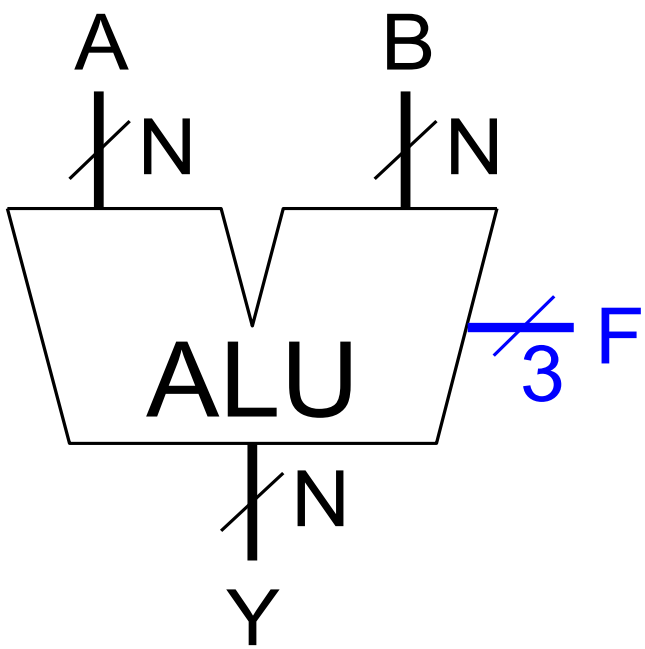
© *Digital Design and Computer Architecture*, 2nd Edition, 2012

Comparator: Less Than



© *Digital Design and Computer Architecture*, 2nd Edition, 2012

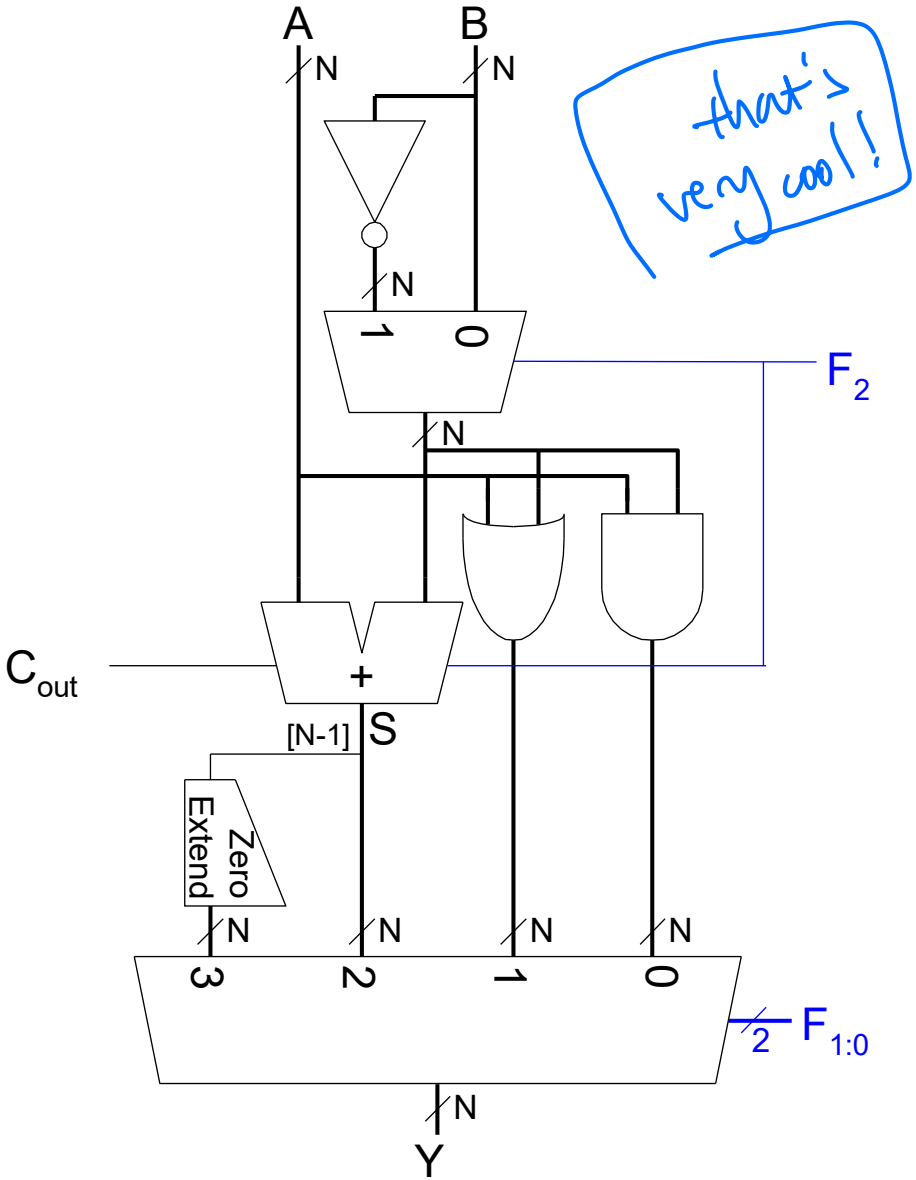
Arithmetic Logic Unit (ALU)



$F_{2:0}$	Function
000	A & B
001	A | B
010	A + B
011	not used
100	A & ~B
101	A | ~B
110	A - B
111	SLT

© *Digital Design and Computer Architecture*, 2nd Edition, 2012

ALU Design

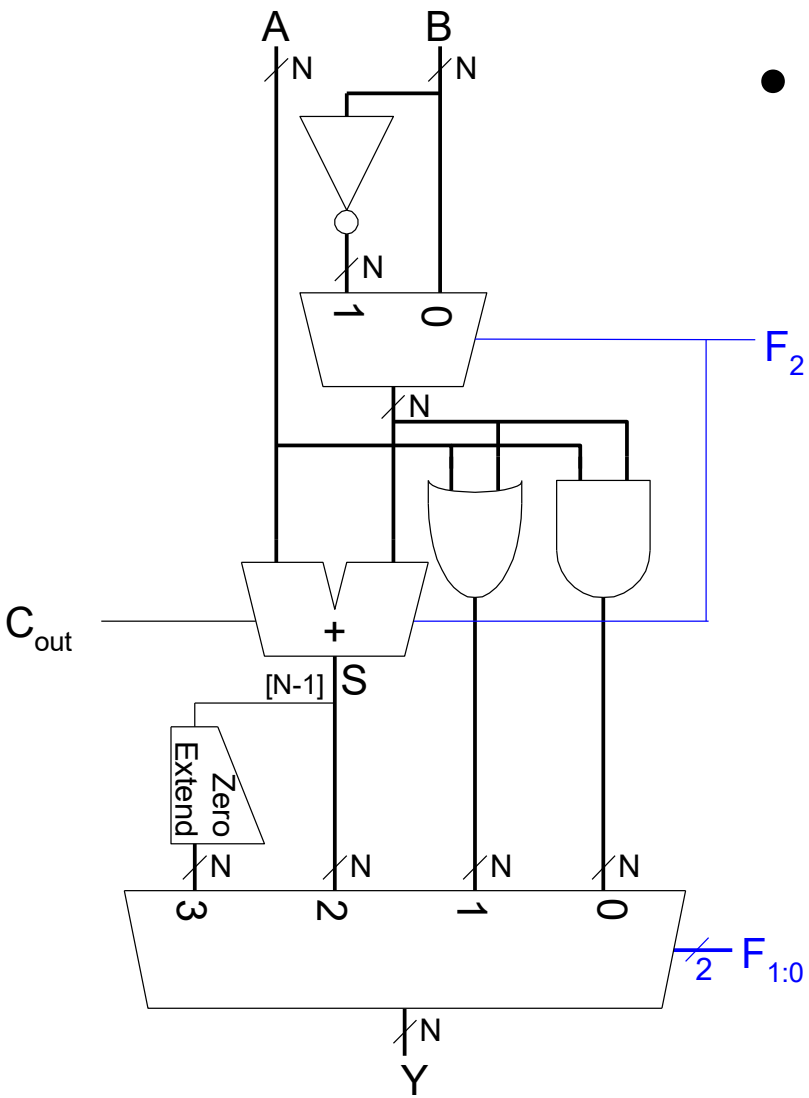


$F_{2:0}$	Function
000	A & B
001	A | B
010	A + B
011	not used
100	A & ~B
101	A | ~B
110	A - B
111	SLT

© *Digital Design and Computer Architecture*, 2nd Edition, 2012

ELSEVIER

Set Less Than (SLT) Example

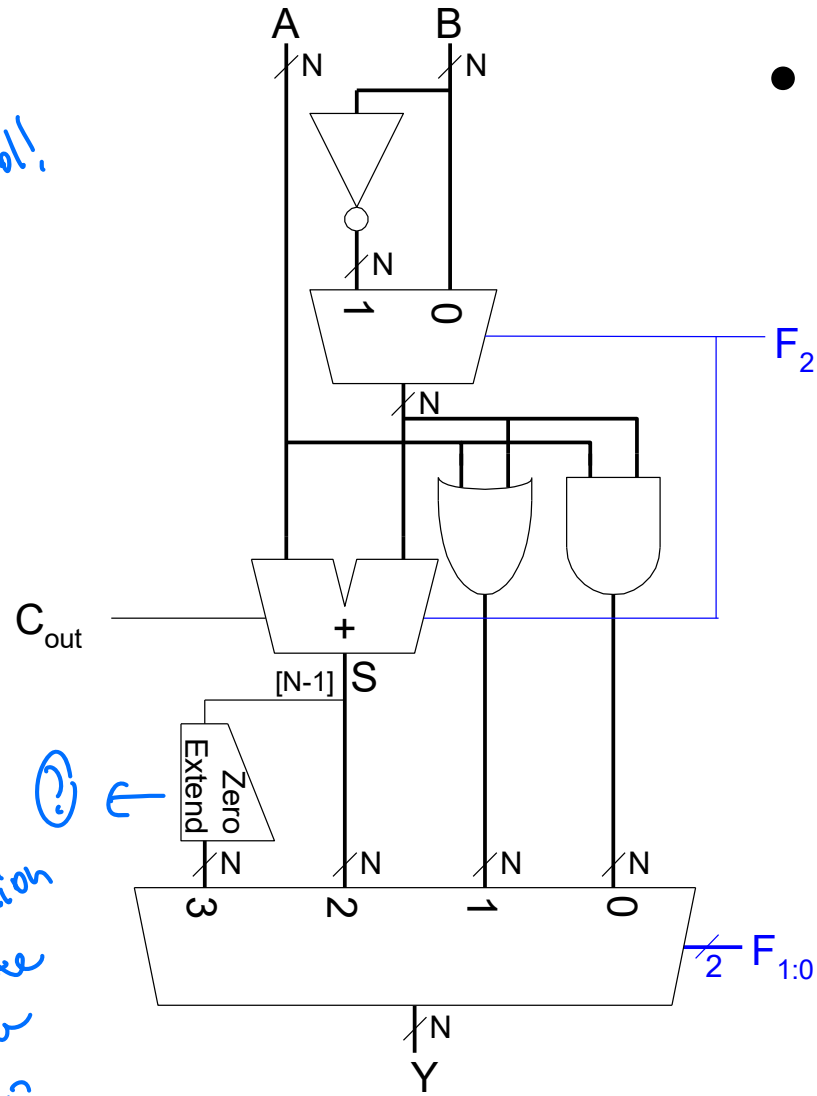


- Configure 32-bit ALU for SLT operation: *A* = 25 and *B* = 32

Copyright © 2007 Elsevier

Set Less Than (SLT) Example

this is very cool!



Isn't this the pad function we wrote the HW?

- Configure 32-bit ALU for SLT operation: A = 25 and B = 32
  - ***A* < *B***, so Y should be 32-bit representation of 1 (0x00000001)
  - $\boldsymbol{F_{2:0}}$ **= 111**
    - $\boldsymbol{F_2}$ **= 1** (adder acts as subtracter), so 25 - 32 = -7
    - -7 has 1 in the most significant bit (S_{31} = 1)
    - $\boldsymbol{F_{1:0}}$ **= 11** multiplexer selects Y = S_{31} (zero extended) = 0x00000001.

© *Digital Design and Computer Architecture*, 2nd Edition, 2012

ELSEVIER

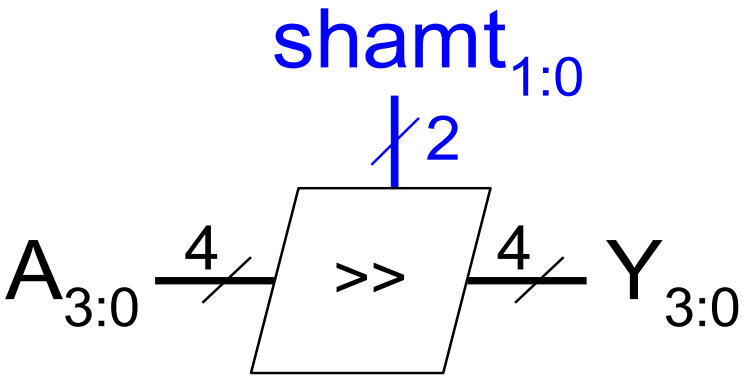
Shifters

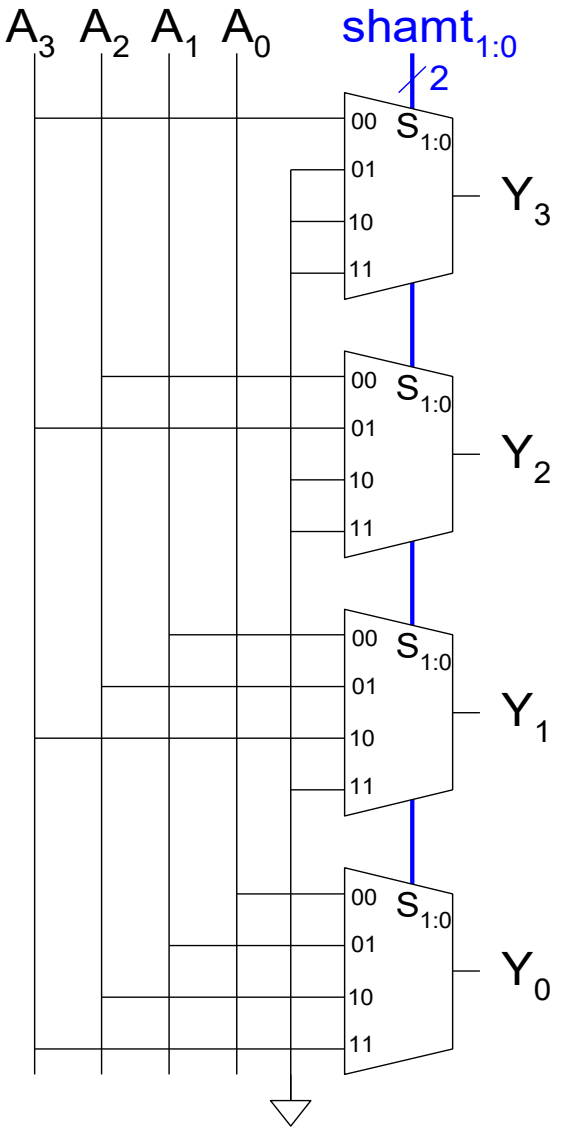
- **Logical shifter:**
  - Ex: **11001** >> 2 = **00110**
  - Ex: **11001** << 2 = **00100**
- **Arithmetic shifter:**
  - Ex: **11001** >>> 2 = **11110**
  - Ex: **11001** <<< 2 = **00100**
- **Rotator:**
  - Ex: **11001** ROR 2 = **01110**
  - Ex: **11001** ROL 2 = **00111**

© *Digital Design and Computer Architecture*, 2nd Edition, 2012

ELSEVIER

Shifter Design





© *Digital Design and Computer Architecture*, 2nd Edition, 2012

ELSEVIER

Shifters as Multipliers, Dividers

- $A << N = A \times 2^N$
  - **Example:** 00001 << 2 = 00100 ($1 \times 2^2 = 4$)
  - **Example:** 11101 << 2 = 10100 ($-3 \times 2^2 = -12$)
- $A >>> N = A \div 2^N$
  - **Example:** 01000 >>> 2 = 00010 ($8 \div 2^2 = 2$)
  - **Example:** 10000 >>> 2 = 11100 ($-16 \div 2^2 = -4$)

© *Digital Design and Computer Architecture*, 2nd Edition, 2012

ELSEVIER

Multipliers

- **Partial products** formed by multiplying a single digit of the multiplier with multiplicand
- **Shifted** partial products **summed** to form result

Decimal		**Binary**
230	multiplicand	0101
x 42	multiplier	x 0111
460	partial products	0101
+ 920		0101
		0101
		+ 0000
9660	result	0100011

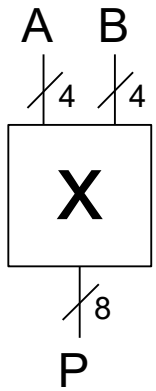
230 x 42 = 9660

5 x 7 = 35

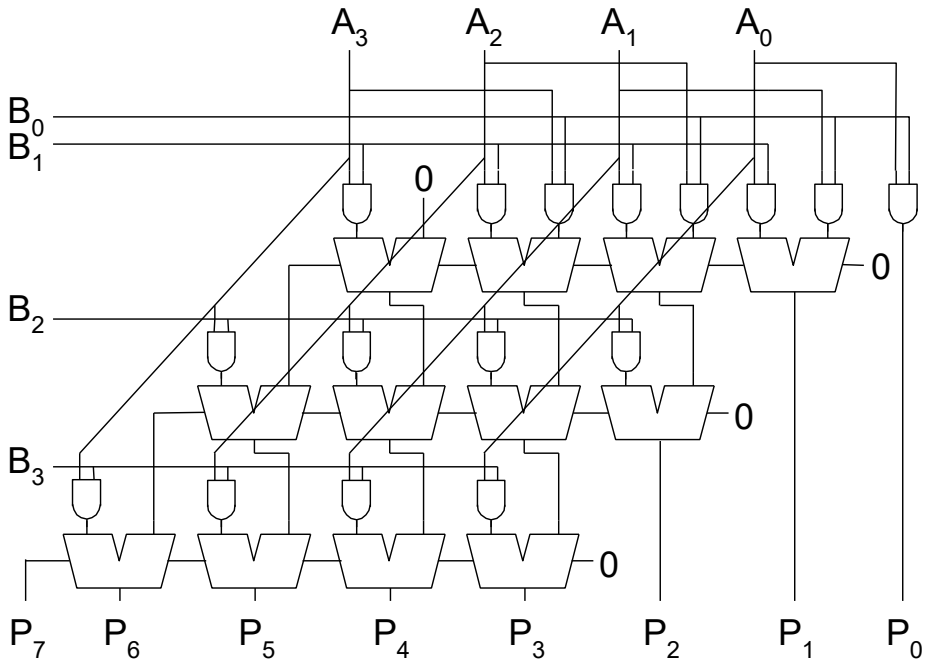
© *Digital Design and Computer Architecture*, 2nd Edition, 2012

ELSEVIER

4 x 4 Multiplier



$$
\begin{array}{rcccccccc}
 & & & & A_3 & A_2 & A_1 & A_0 \\
 & & & \times & B_3 & B_2 & B_1 & B_0 \\
\hline
 & & & & A_3B_0 & A_2B_0 & A_1B_0 & A_0B_0 \\
 & & & A_3B_1 & A_2B_1 & A_1B_1 & A_0B_1 & \\
 & & A_3B_2 & A_2B_2 & A_1B_2 & A_0B_2 & & \\
+ & A_3B_3 & A_2B_3 & A_1B_3 & A_0B_3 & & & \\
\hline
P_7 & P_6 & P_5 & P_4 & P_3 & P_2 & P_1 & P_0
\end{array}
$$



© *Digital Design and Computer Architecture*, 2nd Edition, 2012

ELSEVIER